

UW PICO 5.09

File: integrated_shell.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <errno.h>
#include <signal.h>
#include <sys/types.h>
#include <sys/wait.h>

#define MAX_ARGS 32
#define BUF_SIZE 1024
#define LOG_FILE "shell_errors.log"

// Log errors to both stderr and a persistent log file
void log_error(const char *category, const char *message) {
    fprintf(stderr, "[ERROR %s]: %s\n", category, message);
    FILE *log_fp = fopen(LOG_FILE, "a");
    if (log_fp) {
        fprintf(log_fp, "[%s] %s\n", category, message);
        fclose(log_fp);
    }
}

// Trim leading and trailing whitespace
char *trim_space(char *str) {
    while (*str == ' ' || *str == '\t') str++;
    if (*str == 0) return str;
    char *end = str + strlen(str) - 1;
    while (end > str && (*end == ' ' || *end == '\t')) end--;
    end[1] = '\0';
    return str;
}

// Validate input syntax prior to process execution
int validate_syntax(const char *cmd) {
    int quote_count = 0;
    for (int i = 0; cmd[i] != '\0'; i++) {
        if (cmd[i] == '"' || cmd[i] == '\'') {
            quote_count++;
        }
    }
    if (quote_count % 2 != 0) {
        log_error("Syntax Error", "Unmatched quotes detected in command.");
        return 0; // Invalid syntax
    }
    return 1; // Valid syntax
}
```

^G Get Help
^X Exit

^O WriteOut
^J Justify

^R Read File
^W Where is

^Y Prev Pg
^V Next Pg

^K Cut Text
^U UnCut Tex

UW PICO 5.09

File: integrated_shell.c

```
char *trim_space(char *str) {
    while (*str == ' ' || *str == '\t') str++;
    if (*str == 0) return str;
    char *end = str + strlen(str) - 1;
    while (end > str && (*end == ' ' || *end == '\t')) end--;
    end[1] = '\0';
    return str;
}

// Validate input syntax prior to process execution
int validate_syntax(const char *cmd) {
    int quote_count = 0;
    for (int i = 0; cmd[i] != '\0'; i++) {
        if (cmd[i] == '"' || cmd[i] == '\'') {
            quote_count++;
        }
    }
    if (quote_count % 2 != 0) {
        log_error("Syntax Error", "Unmatched quotes detected in command.");
        return 0; // Invalid syntax
    }
    return 1; // Valid syntax
}

/
Parse input string into argument tokens
int parse_input(char *line, char **args) {
    int arg_count = 0;
    char *token = strtok(line, " \t");
    while (token != NULL && arg_count < MAX_ARGS - 1) {
        args[arg_count++] = token;
        token = strtok(NULL, " \t");
    }
    args[arg_count] = NULL;
    return arg_count;
}

// Execute command with end-to-end runtime error handling
void execute_command(char *line) {
    line = trim_space(line);
    if (strlen(line) == 0) return;

    // Validate syntax before proceeding
    if (!validate_syntax(line)) return;

    char *args[MAX_ARGS];
    int arg_count = parse_input(line, args);
    if (arg_count == 0) return;
```

^G Get Help
^X Exit

^O WriteOut
^J Justify

^R Read File
^W Where is

^Y Prev Pg
^V Next Pg

^K Cut Text
^U UnCut Tex

```
~ --zsh ... | ~ -- nano memory_and_test_shell.c
UW PICO 5.09 File: integrated_shell.c

/
/ Parse input string into argument tokens
int parse_input(char *line, char **args) {
    int arg_count = 0;
    char *token = strtok(line, " \t");
    while (token != NULL && arg_count < MAX_ARGS - 1) {
        args[arg_count++] = token;
        token = strtok(NULL, " \t");
    }
    args[arg_count] = NULL;
    return arg_count;
}

// Execute command with end-to-end runtime error handling
void execute_command(char *line) {
    line = trim_space(line);
    if (strlen(line) == 0) return;

    // Validate syntax before proceeding
    if (!validate_syntax(line)) return;

    char *args[MAX_ARGS];
    int arg_count = parse_input(line, args);
    if (arg_count == 0) return;

    pid_t pid = fork();
    if (pid < 0) {
        log_error("Runtime Error", "Fork failed to create child process.");
        return;
    } else if (pid == 0) {
        // Child execution
        if (execvp(args[0], args) < 0) {
            char err_msg[BUF_SIZE];
            snprintf(err_msg, sizeof(err_msg), "Failed to execute '%s': %s", args[0], strerror(errno));
            log_error("Execution Failure", err_msg);
            exit(EXIT_FAILURE);
        }
    } else {
        // Parent process waits for foreground task completion
        int status;
        if (waitpid(pid, &status, 0) < 0) {
            log_error("Runtime Error", "waitpid failed.");
        } else if (WIFEXITED(status) && WEXITSTATUS(status) != 0) {
            char err_msg[BUF_SIZE];
            snprintf(err_msg, sizeof(err_msg), "Command '%s' exited with error code %d.", args[0], WEXITSTATUS(status));
            log_error("Child Non-Zero Exit", err_msg);
        }
    }
}

^G Get Help      ^O WriteOut      ^R Read File      ^Y Prev Pg      ^K Cut Text
^X Exit          ^J Justify       ^W Where is       ^V Next Pg      ^U UnCut Tex
```

```
~ --zsh ... | ~ -- nano memory_and_test_shell.c
UW PICO 5.09 File: integrated_shell.c

pid_t pid = fork();
if (pid < 0) {
    log_error("Runtime Error", "Fork failed to create child process.");
    return;
} else if (pid == 0) {
    // Child execution
    if (execvp(args[0], args) < 0) {
        char err_msg[BUF_SIZE];
        snprintf(err_msg, sizeof(err_msg), "Failed to execute '%s': %s", args[0], strerror(errno));
        log_error("Execution Failure", err_msg);
        exit(EXIT_FAILURE);
    }
} else {
    // Parent process waits for foreground task completion
    int status;
    if (waitpid(pid, &status, 0) < 0) {
        log_error("Runtime Error", "waitpid failed.");
    } else if (WIFEXITED(status) && WEXITSTATUS(status) != 0) {
        char err_msg[BUF_SIZE];
        snprintf(err_msg, sizeof(err_msg), "Command '%s' exited with error code %d.", args[0], WEXITSTATUS(status));
        log_error("Child Non-Zero Exit", err_msg);
    }
}

}

int main() {
    char input_buf[BUF_SIZE];

    // Ignore SIGINT in parent shell to maintain continuous interaction
    signal(SIGINT, SIG_IGN);

    while (1) {
        printf("IntegratedShell > ");
        fflush(stdout);

        if (!fgets(input_buf, sizeof(input_buf), stdin)) {
            // Recover gracefully from EOF or reading errors
            if (feof(stdin)) break;
            log_error("Input Error", "Failed to read line from stdin.");
            clearerr(stdin);
            continue;
        }

        input_buf[strcspn(input_buf, "\n")] = 0; // Strip newline
        if (strcmp(input_buf, "exit") == 0) break;
        if (strlen(input_buf) == 0) continue;
    }
}

^G Get Help      ^O WriteOut      ^R Read File      ^Y Prev Pg      ^K Cut Text
^X Exit          ^J Justify       ^W Where is       ^V Next Pg      ^U UnCut Tex
```

UW PICO 5.09

File: integrated_shell.c

```
}

int main() {
    char input_buf[BUF_SIZE];

    // Ignore SIGINT in parent shell to maintain continuous interaction
    signal(SIGINT, SIG_IGN);

    while (1) {
        printf("IntegratedShell > ");
        fflush(stdout);

        if (!fgets(input_buf, sizeof(input_buf), stdin)) {
            // Recover gracefully from EOF or reading errors
            if (feof(stdin)) break;
            log_error("Input Error", "Failed to read line from stdin.");
            clearerr(stdin);
            continue;
        }

        input_buf[strcspn(input_buf, "\n")] = 0; // Strip newline
        if (strcmp(input_buf, "exit") == 0) break;
        if (strlen(input_buf) == 0) continue;

        execute_command(input_buf);
    }

    return 0;
}
```

^G Get Help
^X Exit

^O WriteOut
^J Justify

^R Read File
^W Where is

^Y Prev Pg
^V Next Pg

^K Cut Text
^U UnCut Tex